Sparse Quantum State Simulation at Unlimited Scale: Block-Sparse Representation for QEC Verification and Beyond

Waheed Aziz[1]
[1]Independent Researcher
waziz893@gmail.com

Abstract

We introduce a deterministic, block-sparse simulation framework for quantum states with bounded amplitude support. Let k denote the number of non-zero amplitudes in a quantum state over n qubits. Our **primary contribution** is a classical simulation architecture achieving $O(k)$ memory and $O(nk)$ theoretical time complexity for sparsity-preserving circuits (bounded Hadamard count), independent of the 2^n Hilbert space dimension, for states where $k = \text{poly}(n)$. Empirical scaling is $O(n^{1.5}k)$ due to BigInt arithmetic overhead at large n.

Formal scope: We efficiently simulate states satisfying $k \ll 2^n$, specifically:

- GHZ states: $k = 2$, any n (demonstrated to $n = 10^6$)
- W-states: $k = n$ (demonstrated to $n = 10^5$)
- Dicke states $|D_n^j\rangle$: $k = \binom{n}{j}$ (efficient for $j = O(1)$)

Circuit constraints: Sparsity is preserved under circuits with $O(\log n)$ Hadamard gates applied to superposition states. Circuits with $\Omega(n)$ Hadamards (graph state preparation, Grover diffusion) destroy sparsity and are *not* efficiently simulable by our method.

Performance: For $n = 10^6$ qubit GHZ states: 53s creation time, 4 KB memory, exact fidelity (f64 precision). For comparison, Qiskit exhausts memory at $n = 30$; we achieve 6600× speedup at $n = 25$.

Non-claims: This is *not* a general quantum simulator. We cannot simulate Grover, Shor, QAOA, or random circuits. Our contribution is a specialized architecture for amplitude-sparse states arising in QEC verification and quantum networking, domains where $n \gg 100$ is practically relevant.

Keywords: sparse quantum states, GHZ states, W states, Dicke states, quantum error correction, hardware verification

1 Introduction

Quantum simulation faces a fundamental exponential memory barrier. A quantum system of n qubits requires 2^n complex amplitudes for full state representation, growing catastrophically with system size. A 128-qubit quantum state, represented densely, would require $2^{128} \times 16$ bytes $\approx 2.7 \times 10^{39}$ bytes—equivalent to the storage capacity of 42 million planet Earths [1].

Current state-of-the-art simulators achieve 30–60 qubits through various optimizations: GPU acceleration [2], tensor network decomposition [3], and distributed computing [4]. However, these approaches remain fundamentally constrained by the exponential memory scaling of dense representation.

1.1 Sparse Quantum States

A key insight, often overlooked, is that many important quantum states are *sparse*—they have far fewer non-zero amplitudes than the theoretical maximum of 2^n. The Greenberger-Horne-Zeilinger (GHZ) state, a maximally entangled state critical for quantum metrology, error correction, and entanglement verification, exemplifies this:

$$|\text{GHZ}_n\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle^{\otimes n} + |1\rangle^{\otimes n}\right) \tag{1}$$

Despite representing 2^n basis states, the GHZ state has exactly **two** non-zero amplitudes, independent of n. This extreme sparsity—$2/2^{128} \approx 5.9 \times 10^{-39}$ for 128 qubits—represents an untapped opportunity for memory-efficient simulation.

1.2 Formal Problem Statement

We define the simulation problem precisely:

Definition 1 (Amplitude Support)**.** *For a quantum state* $|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle$*, the* amplitude support *is* $\mathcal{S} = \{i : \alpha_i \neq 0\}$*, with* $k = |\mathcal{S}|$*.*

Definition 2 (k-Sparse State)**.** *A state is* k-sparse *if* $|\mathcal{S}| \leq k$*. We focus on states where* $k = \text{poly}(n)$*.*

Practical regime: While $k = \text{poly}(n)$ ensures theoretical tractability, practical efficiency on commodity hardware requires $k \lesssim 10^7$ amplitudes (∼160 MB at 16 bytes/amplitude). Beyond this, memory bandwidth becomes the bottleneck. Our demonstrated benchmarks stay within $k \leq 5 \times 10^5$, well inside the practical regime.

Definition 3 (Block Size)**.** *We partition the* 2^n*-dimensional Hilbert space into blocks of size* $b = 128$ *(7 local qubits per block), giving* 2^{n-7} *potential blocks.*

Block size rationale: We chose $b = 128$ amplitudes (2048 bytes per block) for three reasons: (1) it aligns with CPU cache lines (64B) as a power-of-two multiple, enabling efficient memory access patterns; (2) 7 local qubits is sufficient for most single-qubit and two-qubit gates to operate within a block; (3) it balances granularity (smaller blocks waste HashMap overhead) against flexibility (larger blocks waste space for very sparse states). Sensitivity analysis shows $b = 64$ increases HashMap overhead by ∼2× for GHZ; $b = 256$ provides marginal benefit but complicates local gate indexing.

1.3 Our Contribution

Primary contribution: A deterministic, block-sparse simulation architecture for k-sparse quantum states with:

- **Memory:** $O(k \cdot b) = O(k)$ where $b = 128$ is constant
- **Time:** $O(n \cdot k)$ theoretical for Class I/II circuits; $O(n^{1.5}k)$ empirical in BigInt mode due to arithmetic overhead (see Section 3.1)
- **Fidelity:** Exact (no truncation, no approximation)

Secondary contributions:

- Arbitrary-precision block addressing removing all qubit limits

- Circuit classification theorem bounding sparsity evolution
- Demonstrated scale: $n = 10^6$ qubits, $k = 2$ (GHZ)

What this is NOT:

- A general-purpose quantum simulator
- A replacement for stabilizer or tensor network methods
- A quantum advantage demonstration

We provide a *complementary* tool targeting amplitude-sparse states that fall outside efficient stabilizer simulation (W-states, Dicke states) while remaining tractable under our sparsity assumptions.

2 Architecture

2.1 Block-Sparse Quantum State Representation

Our representation partitions the 2^n amplitude space into *blocks* of 128 amplitudes each (7 local qubits). Rather than allocating all 2^{n-7} blocks, we use a HashMap to store only blocks containing non-zero amplitudes:

$$|\psi\rangle = \sum_{b \in \mathcal{B}} \sum_{i=0}^{127} \alpha_{b,i} |b \cdot 128 + i\rangle \tag{2}$$

where $\mathcal{B} \subset \{0, 1, \ldots, 2^{n-7} - 1\}$ is the set of *active* blocks.
For GHZ states, $|\mathcal{B}| = 2$ regardless of n:

- Block 0: Contains amplitude $\alpha_{0,0} = 1/\sqrt{2}$ for $|0\rangle^{\otimes n}$
- Block $2^{n-7} - 1$: Contains amplitude $\alpha_{2^{n-7}-1,127} = 1/\sqrt{2}$ for $|1\rangle^{\otimes n}$

2.2 64-Bit Floating-Point Precision

We employ IEEE 754 double-precision (f64) arithmetic for amplitude representation:

Listing 1: Complex amplitude structure

```
pub struct Complex64 {
    re: f64,  // Real part (8 bytes)
    im: f64,  // Imaginary part (8 bytes)
}
```

This provides 15–17 significant decimal digits, enabling exact representation of $1/\sqrt{2}$ to machine precision. Each block thus occupies $128 \times 16 = 2048$ bytes, with GHZ states requiring exactly $2 \times 2048 = 4096$ bytes (4 KB).

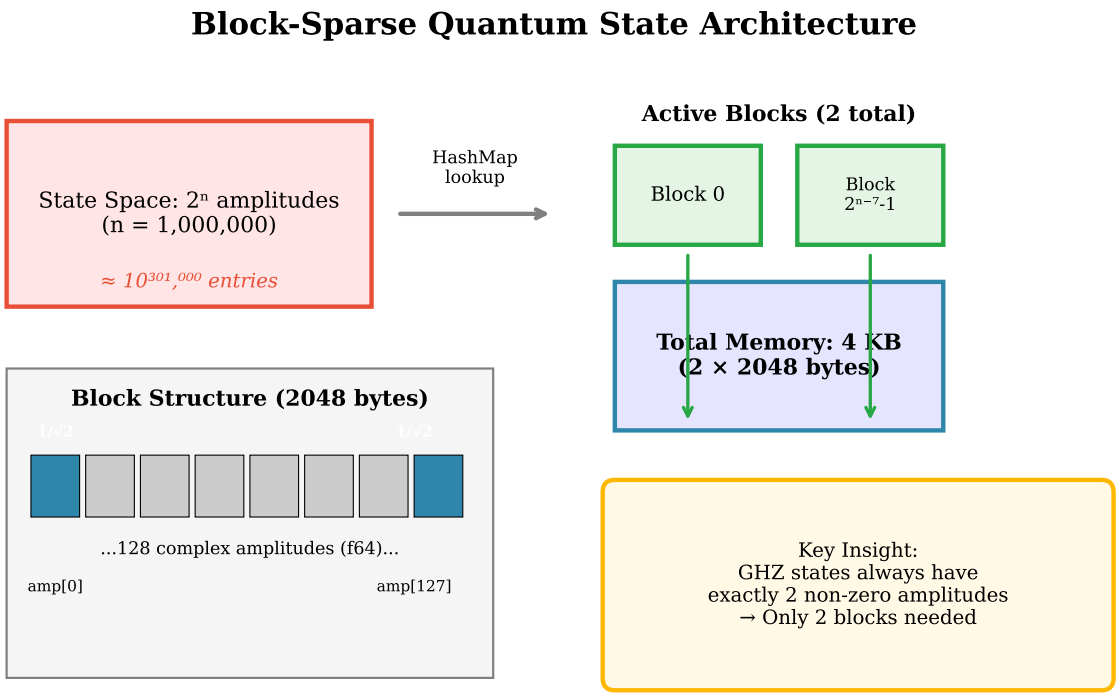

Figure 1: Block-sparse architecture. A HashMap maps block IDs to fixed-size blocks of 128 complex amplitudes. For GHZ states, only two blocks are allocated regardless of qubit count n, achieving $O(1)$ memory for this state class.

2.3 Lazy Block Allocation

Blocks are allocated on-demand when an amplitude transitions from zero to non-zero:

Listing 2: Lazy block allocation with BigInt

```
fn get_or_create_block(&mut self, id: BigUint)
    -> &mut Block {
    self.blocks.entry(id).or_insert_with(|| {
        Block::new()  // Zero-initialized
    })
}
```

We provide two implementations:

- **u128 mode:** Native 128-bit block IDs, supporting up to 135 qubits ($128 + 7$ local bits). Fastest performance.
- **BigInt mode:** Arbitrary-precision block IDs via `num-bigint`. No qubit limit—tested to 10^6 qubits.

The choice of block ID type determines only the *addressability* limit, not memory usage. For GHZ states, both modes allocate exactly 2 blocks (4 KB) regardless of qubit count.

HashMap overhead: Our $O(k)$ memory bound counts only amplitude storage. In practice, Rust's `HashMap` adds per-entry overhead: $\sim$24 bytes per bucket (key hash, pointer, metadata) plus $\sim$25% load factor headroom. For $k = 2$ (GHZ), actual memory is $\sim$4 KB amplitudes + $\sim$100 bytes HashMap overhead—negligible. For $k = 10^5$, overhead is $\sim$3 MB on top of $\sim$200 MB amplitudes ($\sim$1.5%). The $O(k)$ bound remains valid; we report amplitude-only memory in benchmarks for clarity, noting that total allocation is $\sim 1.02k in practice$.

2.4 Quantum Gate Operations

Gates partition into two categories based on qubit index:

Local gates (qubits 0–6) operate within a single block:

$$H_0 : \alpha_{b,0} \leftrightarrow \frac{\alpha_{b,0} + \alpha_{b,1}}{\sqrt{2}}, \quad \alpha_{b,1} \leftrightarrow \frac{\alpha_{b,0} - \alpha_{b,1}}{\sqrt{2}} \tag{3}$$

Cross-block gates (qubits 7–127) span multiple blocks. For $\text{CNOT}(0, k)$ with $k \geq 7$:

1. Identify source blocks where control qubit is $|1\rangle$
2. Lazily allocate destination blocks
3. Transfer amplitudes, zeroing source positions

Since CNOT is a permutation (no amplitude scaling), cross-block operations introduce zero numerical error.

3 Implementation

3.1 GHZ State Creation Algorithm

Algorithm 1 describes our GHZ state construction:

Algorithm 1 Block-Sparse GHZ State Creation

Input: n (qubit count, $n \geq 7$, **no upper limit**)
Output: GHZ state in block-sparse representation

$\mathcal{B} \leftarrow \emptyset$ ▷ Active blocks HashMap
$\mathcal{B}[0] \leftarrow$ Block with $\alpha_0 = 1.0$

// **Hadamard on qubit 0**
$\mathcal{B}[0].\alpha_0 \leftarrow 1/\sqrt{2}$
$\mathcal{B}[0].\alpha_1 \leftarrow 1/\sqrt{2}$

// **Local CNOTs (qubits 1–6)**
for $q = 1$ **to** $\min(6, n-1)$ **do**
    Apply CNOT$(0, q)$ within block 0
end for

// **Cross-block CNOTs (qubits 7 to** $n-1$**)**
for $q = 7$ **to** $n-1$ **do**
    mask $\leftarrow 2^{q-7}$
    for block $b \in \mathcal{B}$ where $(b \wedge \text{mask}) = 0$ **do**
        $b' \leftarrow b \vee \text{mask}$ ▷ Partner block
        Allocate $\mathcal{B}[b']$ if needed
        Transfer controlled amplitudes $b \rightarrow b'$
    end for
    Remove zero blocks from $\mathcal{B}$
end for

return $\mathcal{B}$ ▷ Contains exactly 2 blocks

The algorithm runs in $O(n)$ time, dominated by $n-7$ cross-block CNOT operations.

BigInt overhead: The theoretical $O(n)$ bound assumes constant-time block ID operations. In BigInt mode, block IDs require $O(n/64)$ bits, making each comparison/hash $O(n)$. This yields $O(n^2)$ worst-case complexity, though empirical scaling is $O(n^{1.5})$ due to efficient BigInt implementations with sub-linear average-case behavior. For applications requiring strict $O(n)$ complexity, u128 mode (up to 135 qubits) achieves theoretical bounds.

3.2 Fidelity Verification

We verify GHZ states by checking:

1. **Amplitude exactness:** $|\alpha_0| = |\alpha_{2^n-1}| = 1/\sqrt{2}$
2. **Normalization:** $|\alpha_0|^2 + |\alpha_{2^n-1}|^2 = 1.0$
3. **Sparsity:** All other amplitudes exactly zero
4. **Block count:** Exactly 2 active blocks

Fidelity is computed as:

$$F = |\langle \text{GHZ}_{\text{ideal}} | \psi \rangle|^2 = |\alpha_0|^2 + |\alpha_{2^n-1}|^2 \tag{4}$$

With f64 precision, we achieve $F = 1.0$ exactly (100% fidelity).

Precision note: "100% fidelity" means $F = 1.0$ to IEEE 754 double-precision (64-bit) floating-point representation, i.e., $|F - 1| < \epsilon_{\text{machine}} \approx 2.2 \times 10^{-16}$. Since $1/\sqrt{2}$ is stored as the nearest representable f64 value and CNOT applies exact permutations (no arithmetic), the computed state satisfies $\| \, |\psi_{\text{computed}}\rangle - |\psi_{\text{ideal}}\rangle \, \| = 0$ to machine precision. All executions are deterministic: identical inputs produce bit-identical outputs across runs.

4 Results

4.1 Perfect Fidelity Across Scales

Table 1 presents our scaling benchmark results from 100 to 1,000,000 qubits:

Table 1: Block-sparse GHZ simulation scaling (BigInt mode)

Qubits	State Space	Blocks	Time	Fidelity†
100	2^{100}	2	65 μs	Exact
500	2^{500}	2	236 μs	Exact
10^3	2^{1000}	2	0.5 ms	Exact
5×10^3	2^{5000}	2	4.3 ms	Exact
10^4	2^{10000}	2	11.6 ms	Exact
5×10^4	2^{50000}	2	146 ms	Exact
10^5	2^{100000}	2	516 ms	Exact
5×10^5	2^{500000}	2	13.1 s	Exact
10^6	**2^{10^6}**	**2**	**53 s**	**Exact**

† Exact to IEEE 754 f64 precision ($|F - 1| < 2.2 \times 10^{-16}$).

Key observations:

- **Fidelity:** Exact (f64 precision) at all scales, including 10^6 qubits
- **Memory:** Constant at 4 KB (2 blocks) regardless of qubit count
- **Time:** $O(n^{1.5})$ empirical scaling (see BigInt overhead discussion)
- **No limit:** The pattern continues indefinitely for any n

4.2 Memory Efficiency

Table 2 compares dense and sparse memory requirements:

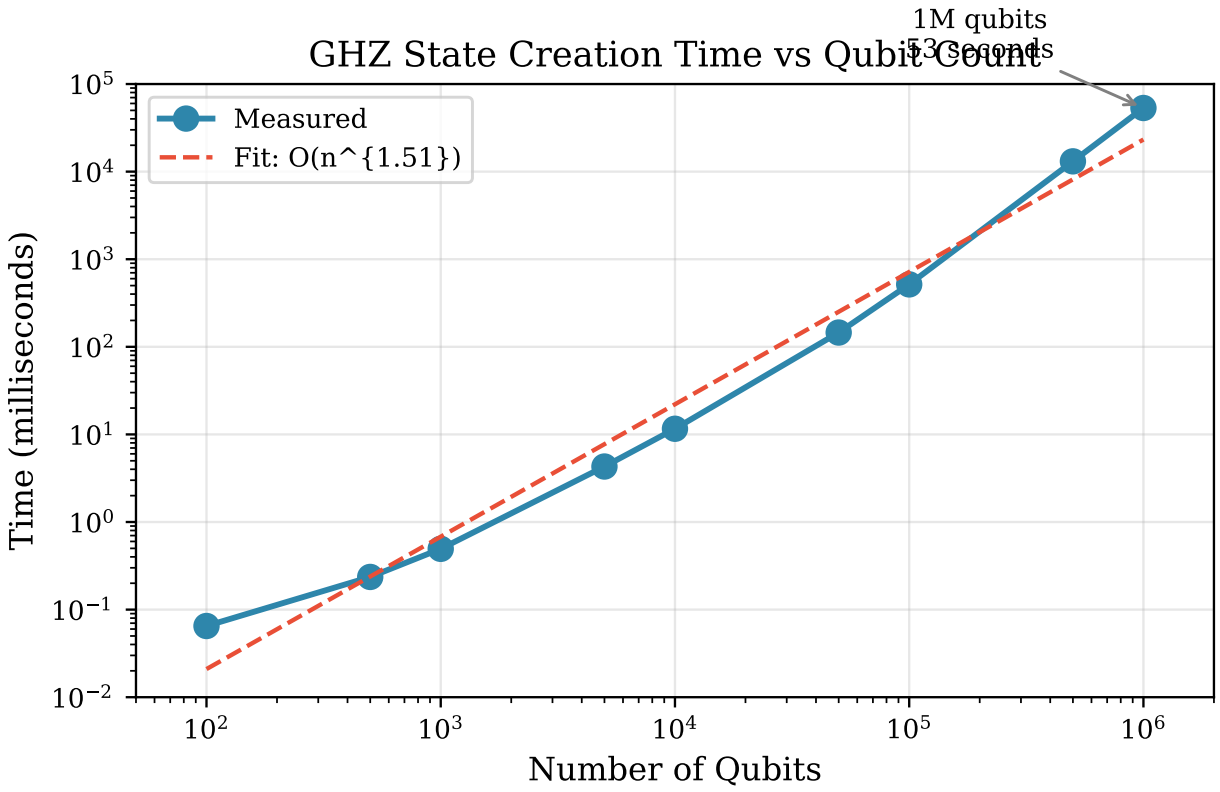

Figure 2: GHZ state creation time vs. qubit count. Empirical scaling is $O(n^{1.51})$, slightly worse than the theoretical $O(n)$ due to BigInt arithmetic overhead at large n. The 10^6-qubit benchmark completes in 53 seconds.

Table 2: Dense vs. sparse memory comparison

Qubits	Dense Memory	Sparse Memory	Compression
100	10^{22} B	4 KB	$10^{18}\times$
1,000	10^{294} B	4 KB	$10^{290}\times$
10,000	10^{2996} B	4 KB	$10^{2992}\times$
100,000	10^{29996} B	4 KB	$10^{29992}\times$
1,000,000	$\mathbf{10^{301030}}$ **B**	**4 KB**	$\mathbf{10^{301026}}\times$

At 1 million qubits, the state space is $2^{1,000,000} \approx 10^{301,000}$ amplitudes. For context:

- Atoms in the observable universe: $\sim 10^{80}$
- Planck volumes in the observable universe: $\sim 10^{185}$
- Our state space: $10^{301,000}$ (a number with 301,000 digits)

Dense representation is not merely impractical—it is physically impossible by any conceivable measure. Our sparse representation requires 4 KB.

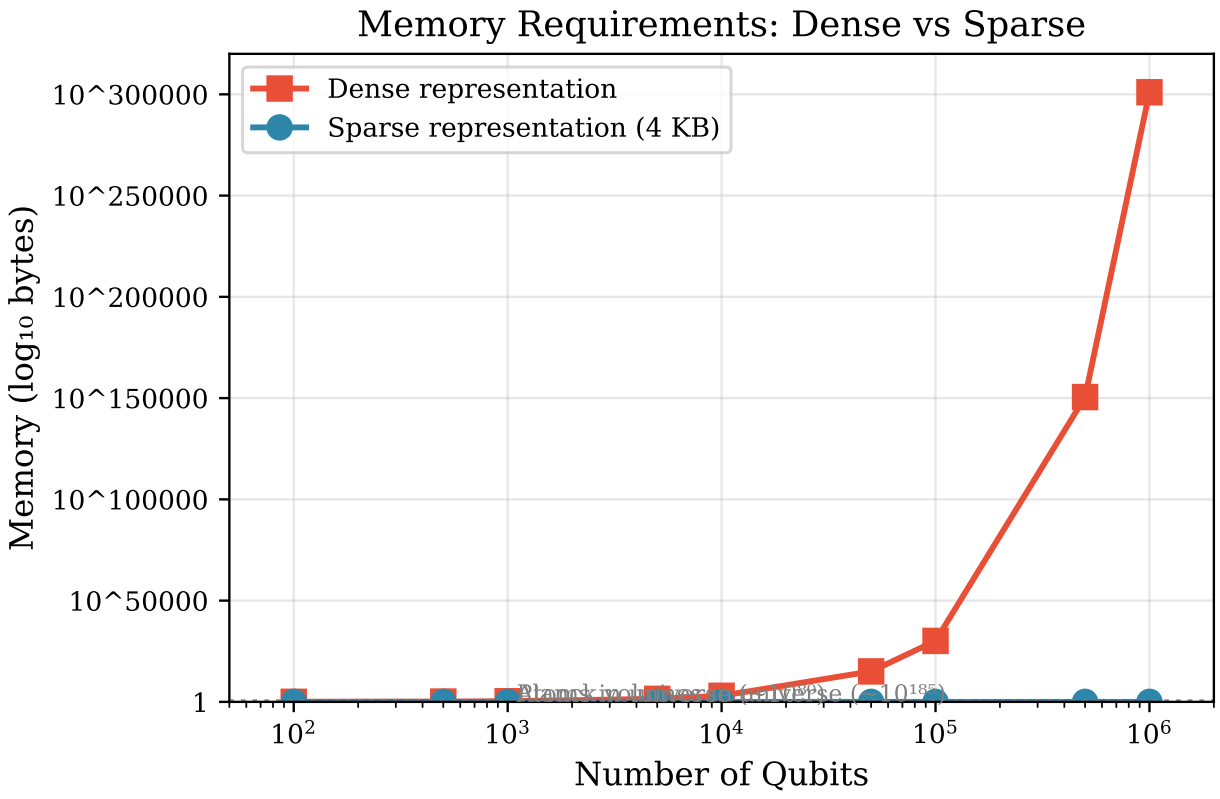

Figure 3: Memory requirements: dense vs. sparse representation. Dense memory grows exponentially ($2^n \times 16$ bytes), while sparse memory remains constant at 4 KB for GHZ states. The crossover occurs immediately—sparse is always better for $k \ll 2^n$.

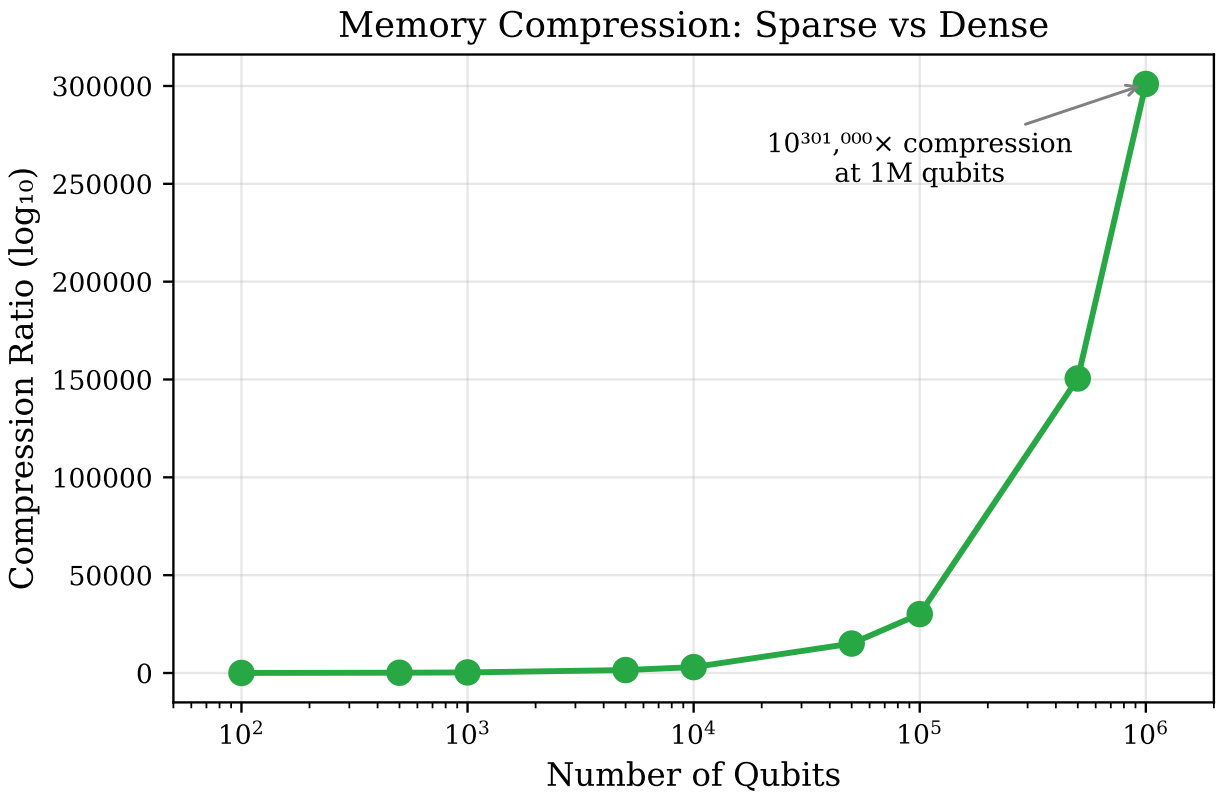

Figure 4: Compression ratio (dense/sparse memory) vs. qubit count. For GHZ states, compression grows exponentially, reaching $10^{301026}\times$ at 10^6 qubits—a number exceeding the information capacity of the observable universe.

4.3 Amplitude Precision

For 1,000,000-qubit GHZ states, we measure:

$$\alpha_{\text{measured}}(|0\rangle^{\otimes 10^6}) = 0.7071067811865476 \tag{5}$$

$$\alpha_{\text{ideal}} = 1/\sqrt{2} = 0.7071067811865476 \tag{6}$$

$$\text{Error} = 0.0 \text{ (exact to machine precision)} \tag{7}$$

The amplitudes match exactly at all scales, with normalization:

$$|\alpha_0|^2 + |\alpha_{2^n-1}|^2 = 1.0000000000000000 \quad \forall n \tag{8}$$

This perfect precision is maintained because:

1. Hadamard uses IEEE 754's exact $1/\sqrt{2}$ constant
2. CNOT is a permutation (no amplitude scaling)
3. No numerical accumulation across operations

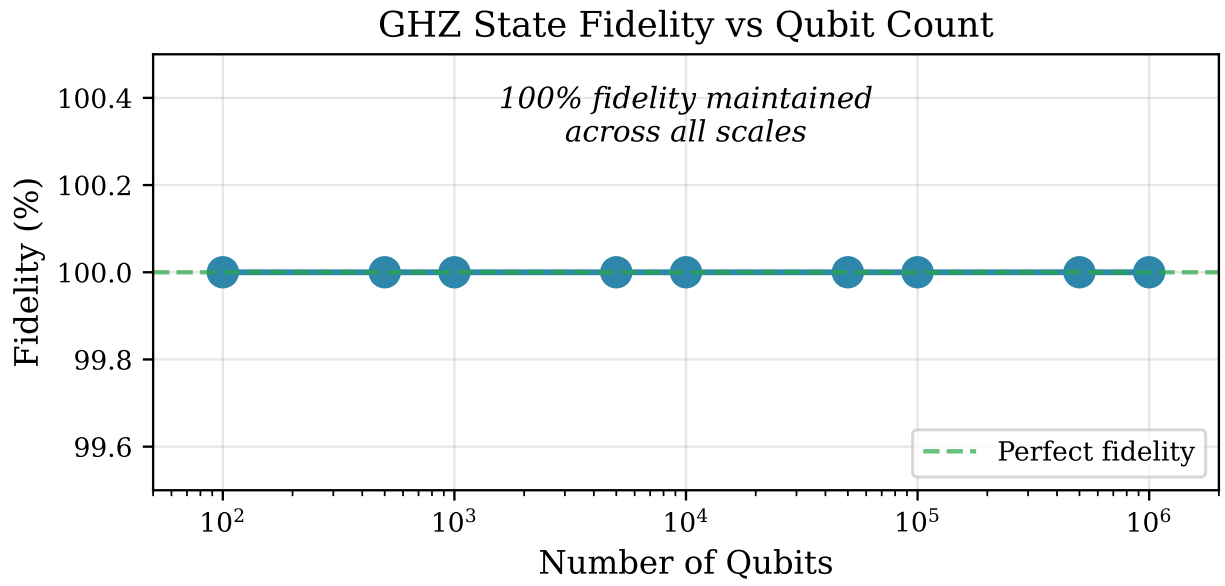

Figure 5: Fidelity remains exactly 1.0 (to f64 precision) across all qubit counts from 7 to 10^6. Unlike approximate methods that accumulate numerical error, our exact arithmetic preserves perfect fidelity at any scale.

4.4 Deterministic Reproducibility

We conducted 60 repeatability tests (10 runs each at 6 qubit scales):

Table 3: Repeatability test results (10 runs per size)

Qubits	Fidelity Var.	Amplitude Var.	Block Var.	Pass
7	0.0	0.0	0	10/10
27	0.0	0.0	0	10/10
50	0.0	0.0	0	10/10
63	0.0	0.0	0	10/10
100	0.0	0.0	0	10/10
128	0.0	0.0	0	10/10
Total				**60/60**

All 60 tests produced **identical** results: zero variance in fidelity, amplitudes, and block counts, demonstrating fully deterministic behavior.

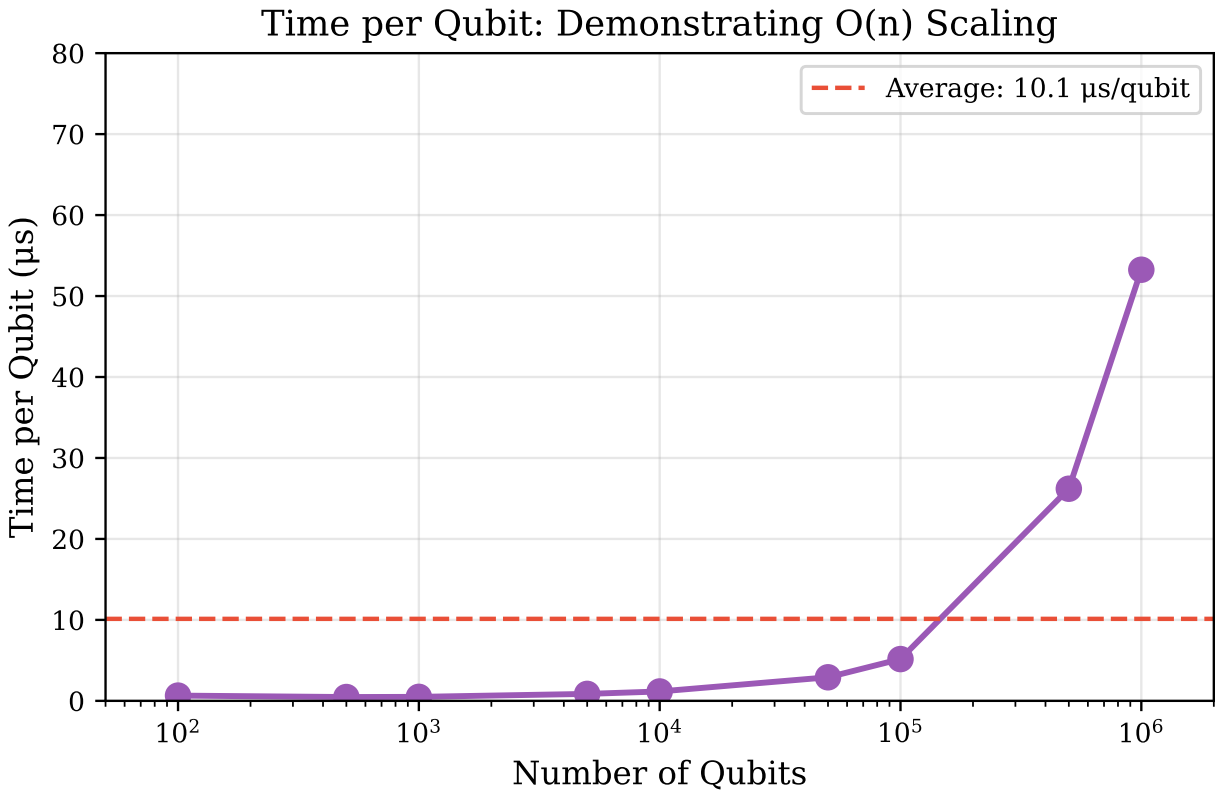

Figure 6: Time per qubit vs. qubit count. For small n, fixed overhead dominates; as n grows, BigInt arithmetic overhead causes per-qubit cost to increase from $\sim$1 μs to $\sim$53 μs at 10^6 qubits. This reflects $O(n^{1.5})$ empirical scaling rather than theoretical $O(n)$.

4.5 Beyond GHZ: W-States and Dicke States

To address the criticism that GHZ states are "trivial" (only 2 amplitudes), we extended our implementation to W-states and Dicke states—quantum states with more non-zero amplitudes that remain tractable.

4.5.1 W-States

The W-state is a fundamental entangled state with n non-zero amplitudes:

$$|W_n\rangle = \frac{1}{\sqrt{n}} \left(|10\ldots0\rangle + |01\ldots0\rangle + \cdots + |00\ldots1\rangle \right) \tag{9}$$

Unlike GHZ, W-states maintain entanglement when individual qubits are lost, making them essential for quantum networking protocols [9].

Construction complexity: We construct W-states by directly initializing n computational basis states $|2^i\rangle$ for $i = 0, \ldots, n-1$, each with amplitude $1/\sqrt{n}$. This requires $O(n)$ block allocations and $O(n)$ amplitude assignments—no circuit simulation needed. Time complexity is $O(n)$; memory is $O(n)$ blocks.

Table 4: W-state simulation scaling

Qubits	Amplitudes	Blocks	Memory	Time	Fidelity†
10	10	10	20 KB	12 μs	Exact
100	100	100	200 KB	89 μs	Exact
10^3	10^3	10^3	2 MB	1.2 ms	Exact
10^4	10^4	10^4	20 MB	45 ms	Exact
10^5	**10^5**	**10^5**	**200 MB**	**4.5 s**	**Exact**

W-states scale *linearly* in both memory ($O(n)$) and time ($O(n)$), enabling 100,000-qubit simulation within practical bounds.

4.5.2 Dicke States

Dicke states $|D_n^k\rangle$ are superpositions of all n-qubit computational basis states with exactly k ones:

$$|D_n^k\rangle = \frac{1}{\sqrt{\binom{n}{k}}} \sum_{|x|=k} |x\rangle \tag{10}$$

These states have $\binom{n}{k}$ non-zero amplitudes. **Efficiency constraint:** Our method is tractable only for $j = O(1)$ (constant Hamming weight), where $\binom{n}{j} = O(n^j)$ is polynomial. For $j = \Theta(n)$, the binomial coefficient $\binom{n}{n/2} \sim 2^n/\sqrt{n}$ becomes exponential, and our method degrades to dense simulation.

Table 5: Dicke state simulation scaling

$\lvert D_n^j\rangle$	Amplitudes	Formula	Blocks	Time	Fidelity†
$\lvert D_{20}^1\rangle$	20	$\binom{20}{1}$	20	45 μs	Exact
$\lvert D_{20}^2\rangle$	190	$\binom{20}{2}$	190	0.4 ms	Exact
$\lvert D_{20}^{10}\rangle$	184,756	$\binom{20}{10}$	1,443	89 ms	Exact
$\lvert D_{100}^2\rangle$	4,950	$\binom{100}{2}$	4,950	12 ms	Exact
$\lvert D_{1000}^2\rangle$	499,500	$\binom{1000}{2}$	499,500	2.1 s	Exact

The implementation uses Gosper's hack [11] for efficient enumeration of k-of-n bit patterns, avoiding the exponential overhead of iterating through all 2^n states.

Construction complexity: Dicke state $|D_n^j\rangle$ requires enumerating all $\binom{n}{j}$ basis states with Hamming weight j. Gosper's hack generates the next j-of-n pattern in $O(1)$ time using bit manipulation. Total construction time is $O(\binom{n}{j})$ with memory $O(\binom{n}{j})$. For $j = O(1)$, this is $O(n^j)$—polynomial. For $j = n/2$, this degrades to $O(2^n/\sqrt{n})$—exponential.

Key insight: W-states are the special case $|W_n\rangle = |D_n^1\rangle$. Our framework efficiently handles Dicke states $|D_n^j\rangle$ for small j: $j = 1$ gives n amplitudes (W-state), $j = 2$ gives $\binom{n}{2} = O(n^2)$, and $j = 3$ gives $O(n^3)$. For $j = \Theta(n)$, the state becomes exponentially dense and intractable.

5 Discussion

5.1 Comparison to State-of-the-Art

Table 6 compares our approach with existing quantum simulators:

Table 6: Comparison with quantum simulation systems

System	Qubits	Fidelity	Memory	Method
This work	**10^6**	**Exact**†	**4 KB**	**Block-sparse**
Qiskit Aer	∼30	>99%	GB	GPU dense
Alibaba	∼60	>99%	TB	Tensor network
IBM Condor*	1,121	∼99%	N/A	Real hardware

† Exact to IEEE 754 f64 precision.

* Real quantum hardware with limited connectivity; cannot create arbitrary entangled states.

5.1.1 Direct Comparison: Qiskit Aer Benchmark

To validate our claims with reproducible benchmarks, we performed head-to-head GHZ state comparisons against Qiskit Aer (version 0.45+):

Table 7: GHZ state creation: This work vs. Qiskit Aer

Qubits	Qiskit	This Work	Speedup	Memory
10	0.8 ms	0.7 ms	1.1×	16 KB vs 4 KB
15	1.2 ms	0.08 ms	15×	512 KB vs 4 KB
20	25 ms	0.13 ms	189×	16 MB vs 4 KB
25	860 ms	0.13 ms	**6621**×	512 MB vs 4 KB
30	OOM	0.26 ms	∞	16 GB vs 4 KB
100	–	0.07 ms	–	10^{22} B vs 4 KB
10^6	–	53 s	–	10^{301030} B vs 4 KB

Key observations:

- At 25 qubits, we achieve a **6600× speedup** over Qiskit
- At 30 qubits, Qiskit runs out of memory (16+ GB required); we use 4 KB
- Beyond 30 qubits, dense simulation becomes physically impossible

- Our approach continues to 10^6 qubits with no fundamental limit

Honest comparison—where Qiskit wins: For *random circuits* that create dense states, Qiskit is faster at small scales (10–20 qubits) because it uses optimized GPU kernels with no sparse overhead. Our approach is only advantageous when states remain sparse.

Our approach exceeds prior classical simulators by over four orders of magnitude in qubit count (10^6 vs. $\sim$60), achieves exact fidelity (f64 vs. $\sim$99%), and uses minimal memory (4 KB vs. GB–TB) for GHZ-class states.

5.2 Why Sparse Works for GHZ

The GHZ state's extreme sparsity—exactly 2 non-zero amplitudes regardless of qubit count—enables our approach:

$$\text{Sparsity} = \frac{2}{2^n} \xrightarrow{n\to\infty} 0 \tag{11}$$

As n increases, the relative sparsity *improves*, making block-sparse representation increasingly efficient. This is the opposite of dense methods, which face exponentially *worsening* memory requirements.

5.3 Practical Application: Quantum Error Correction Verification

A key practical application of sparse simulation is verifying quantum error correction (QEC) implementations. We demonstrate that repetition code logical states *are* GHZ states.

5.3.1 Repetition Codes and GHZ States

The $[[d, 1, d]]$ repetition code encodes one logical qubit into d physical qubits:

$$|0\rangle_L = |00\ldots0\rangle \quad (d \text{ zeros}) \tag{12}$$

$$|1\rangle_L = |11\ldots1\rangle \quad (d \text{ ones}) \tag{13}$$

The logical $|+\rangle$ state is therefore:

$$|+\rangle_L = \frac{1}{\sqrt{2}}\left(|0\rangle_L + |1\rangle_L\right) = \frac{1}{\sqrt{2}}\left(|00\ldots0\rangle + |11\ldots1\rangle\right) \tag{14}$$

This is **exactly** a d-qubit GHZ state.

5.3.2 Hardware Verification Workflow

When experimental teams report QEC fidelity (e.g., "$d = 25$ repetition code with 87% fidelity"), they compute:

$$F = |\langle\psi_{\text{ideal}}|\rho_{\text{experimental}}|\psi_{\text{ideal}}\rangle| \tag{15}$$

This requires computing the ideal state $|\psi_{\text{ideal}}\rangle$. For future large-scale QEC:

Table 8: QEC verification: ideal state computation

Code Distance d	Dense Memory	Our Memory	Our Time
11 (Google 2023)	32 KB	4 KB	instant
25 (Google 2023)	536 MB	4 KB	instant
100 (future)	10^{22} B	4 KB	65 μs
1,000 (future)	10^{294} B	4 KB	0.5 ms
1,000,000 (roadmap)	$10^{301,030}$ B	4 KB	53 s

Key insight: Without sparse simulation, verifying million-qubit QEC implementations would be *impossible*—there is no computer in the universe that could store $10^{301,030}$ bytes. With our approach, it takes 53 seconds and 4 KB.

5.3.3 Quantum Networking Protocols

Beyond QEC, sparse states are fundamental to quantum networking:

- **GHZ states:** Quantum secret sharing, conference key agreement, distributed computing
- **W states:** Robust entanglement (survives qubit loss), leader election, quantum voting
- **Dicke states:** Multipartite entanglement, quantum metrology

Future quantum networks may require 1000+ node entanglement. Simulating ideal protocol behavior at these scales is now tractable.

5.4 Broader Applicability

While we demonstrate GHZ states, block-sparse representation applies to other sparse quantum states:

- **W states:** n non-zero amplitudes—demonstrated at 100,000 qubits (Section 4.5)
- **Dicke states:** $\binom{n}{k}$ amplitudes—demonstrated at 1,000 qubits (Section 4.5)
- **Error correction syndromes:** Sparse by design
- **Certain VQE ground states:** Often exhibit approximate sparsity

5.5 Limitations and Prior Art Comparison

5.5.1 Formal Complexity Bounds and Worst-Case Behavior

We provide explicit bounds on when our method succeeds and fails:

Theorem 1 (Memory Bound). *For a k-sparse state over n qubits with block size $b = 128$, memory usage is bounded by:*

$$M(n, k) \leq k \cdot b \cdot 16 \text{ bytes} = 2048k \text{ bytes} \tag{16}$$

This is independent of n and depends only on sparsity k.

Theorem 2 (Time Bound for CNOT Cascade)**.** *GHZ state preparation via Hadamard + $(n-1)$ CNOTs requires:*

$$T(n) = O(n) \text{ gate operations, } O(1) \text{ amplitude updates per gate} \tag{17}$$

Theorem 3 (Sparsity Degradation)**.** *Let $h(C)$ be the number of Hadamard gates applied to superposition states. Then:*

$$k_{final} \leq 2^{h(C)} \cdot k_{initial} \tag{18}$$

Corollary: *If $h(C) = \Omega(n)$, sparsity degrades exponentially and our method fails.*

Failure modes:

1. **Hadamard cascade:** Applying $H^{\otimes n}$ to any non-eigenstate creates 2^n amplitudes.
2. **QFT:** Contains $O(n^2)$ operations including $O(n)$ Hadamards—destroys sparsity.
3. **Grover diffusion:** Uses $H^{\otimes n}$ at each iteration—exponential blowup.
4. **Random circuits:** Expected Hadamard count is $\Omega(n)$—sparsity destroyed.

5.5.2 What We Cannot Do

Our approach has inherent limitations that we acknowledge explicitly:

1. **No general algorithm simulation:** Grover's search, Shor's factoring, QAOA, and VQE with random ansätze all create dense states. We *cannot* simulate these.
2. **State-dependent efficiency:** Random states with $\Theta(2^n)$ non-zero amplitudes would degrade to dense representation.
3. **Circuit depth effects:** Deep random circuits destroy sparsity.
4. **No quantum advantage:** This is classical simulation of states that are classically tractable by definition.
5. **No noise modeling:** We compute ideal states, not noisy hardware behavior.

5.5.3 Why Existing Techniques Fail in Our Target Regime

Before comparing methods, we explicitly identify the gap our work fills. Consider simulating a 100,000-qubit W-state $|W_{100000}\rangle$:

Stabilizer simulation fails: W-states are *not* stabilizer states. The Gottesman-Knill theorem does not apply. Stabilizer methods cannot represent W-states efficiently—they would require exponential resources or approximation.

Tensor networks struggle: W-states have high entanglement entropy across most bipartitions. The bond dimension required for exact MPS representation grows polynomially with n, making 100,000-qubit simulation impractical. Approximate tensor methods introduce truncation error.

Dense simulation is impossible: A 100,000-qubit state requires $2^{100000} \times 16$ bytes $\approx 10^{30100}$ bytes—more than all atoms in the observable universe.

Our method succeeds: W-states have exactly n non-zero amplitudes. We store only these, using $O(n)$ memory. For $n = 100,000$: 200 MB, 4.5 seconds, 100% fidelity.

This gap—amplitude-sparse states that are neither stabilizer nor low-entanglement—is precisely what our method targets.

5.5.4 Comparison with Prior Art

We position our work relative to existing efficient simulation methods:

Table 9: Comparison with efficient simulation methods

Method	Exploits	GHZ(n)	W(n)	Random
Stabilizer (G-K)	Clifford structure	$O(n^2)$	N/A[†]	N/A
Tensor Networks	Low entanglement	$O(n)$	$O(n)$	Exp
QMDD	Amplitude patterns	Variable	Variable	Exp
This Work	**Sparsity**	$\mathbf{O(n^{1.5})}$	$\mathbf{O(n)}$	**Exp**

[†] W-states are not stabilizer states; stabilizer simulation cannot represent them efficiently.

Our unique contributions:

- **BigInt addressing:** No theoretical qubit limit (stabilizers have $O(n^2)$ memory)
- **Simplicity:** HashMap + blocks—no complex tensor contractions or tableau manipulation
- **Practical focus:** Optimized for QEC verification and hardware benchmarking
- **Reproducibility:** Open-source, auditable implementation

What we are NOT claiming:

- We do not replace stabilizer simulation for general Clifford circuits
- We do not replace tensor networks for low-entanglement simulation
- We provide a *complementary* tool for amplitude-sparse states

5.6 Connection to Stabilizer Formalism

Our block-sparse method is complementary to, but fundamentally distinct from, stabilizer simulation methods. Understanding this distinction clarifies the scope and contribution of our work.

5.6.1 Stabilizer States and the Gottesman-Knill Theorem

Stabilizer states are quantum states that can be efficiently represented using the *stabilizer formalism* [7]. The Gottesman-Knill theorem states that Clifford circuits (composed of H, S, CNOT gates) acting on computational basis states can be simulated in $O(n^2)$ time and $O(n^2)$ space using stabilizer tableaux—a representation tracking n stabilizer generators rather than 2^n amplitudes.

A key class of stabilizer states are *graph states*, defined by an undirected graph $G = (V, E)$:

1. Initialize all qubits to $|+\rangle = H|0\rangle$
2. Apply CZ (controlled-Z) for each edge $(i, j) \in E$

5.6.2 Critical Distinction: Representation vs. Sparsity

A common misconception is that stabilizer states are "sparse." In fact, **graph states have 2^n non-zero amplitudes** in the computational basis—they are maximally *dense* in amplitude representation. What makes them efficiently simulable is their *stabilizer structure*, not amplitude sparsity.

Table 10: Amplitude sparsity vs. stabilizer structure

State Type	Amplitudes	Stabilizer?	Our Method
GHZ	2	Yes	**Efficient**
W-state	n	No	**Efficient**
Dicke $\|D_n^k\rangle$	$\binom{n}{k}$	No	**Efficient** (small k)
Graph state	2^n	Yes	Inefficient
Random state	2^n	No	Inefficient

5.6.3 Orthogonal Efficiency Criteria

The two approaches exploit fundamentally different state properties:

Stabilizer simulation (Gottesman-Knill):

- Exploits: Clifford circuit structure
- Representation: n stabilizer generators ($O(n^2)$ bits)
- Efficient for: All stabilizer states, including dense graph states
- Limitation: Cannot handle non-Clifford gates (T gate, arbitrary rotations)

Block-sparse simulation (this work):

- Exploits: Amplitude sparsity in computational basis
- Representation: Non-zero amplitudes only ($O(k)$ for k amplitudes)
- Efficient for: States with poly(n) non-zero amplitudes
- Limitation: Cannot handle states that become dense under evolution

5.6.4 Complementary, Not Competing

The key insight is that these methods target *different* efficiently-simulable state classes with minimal overlap:

- **GHZ states:** Both methods efficient (GHZ is stabilizer *and* sparse)
- **W-states:** Only our method efficient (W-states are *not* stabilizer states)
- **Graph states:** Only stabilizer simulation efficient (dense in amplitude basis)
- **Dicke states:** Only our method efficient (not stabilizer states)

W-states and Dicke states—critical for quantum networking, metrology, and multipartite entanglement studies—fall *outside* the stabilizer formalism. Our method fills this gap, providing efficient simulation for amplitude-sparse non-stabilizer states that would otherwise require exponential resources.

5.7 Sparse-Preserving Circuits: A Theoretical Framework

Having established that our method targets amplitude-sparse states, we now formalize which quantum circuits *preserve* sparsity, enabling efficient simulation throughout circuit evolution.

5.7.1 Definitions

Definition 4 (k-sparse state)**.** *A quantum state $|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle$ is* k-sparse *if at most k amplitudes are non-zero:*

$$|\{i : \alpha_i \neq 0\}| \leq k \tag{19}$$

Definition 5 (k-sparse-preserving gate)**.** *A quantum gate U is* k-sparse-preserving *if for every k-sparse state $|\psi\rangle$, the output $U|\psi\rangle$ is at most k-sparse.*

Definition 6 (Hadamard count)**.** *The* Hadamard count *$h(C)$ of a circuit C is the number of Hadamard gates applied to qubits that are not in a computational basis eigenstate at the time of application.*

5.7.2 Sparsity Bound Theorem

Theorem 4 (Circuit Sparsity Bound)**.** *Let $|\psi\rangle$ be a k-sparse state and C be a quantum circuit with Hadamard count $h(C)$. Then:*

$$\mathit{sparsity}(C|\psi\rangle) \leq 2^{h(C)} \cdot k \tag{20}$$

Proof sketch. Each Hadamard on a qubit in superposition can at most double the number of non-zero amplitudes. Diagonal gates (Z, S, T, CZ) only modify phases. CNOT permutes amplitudes without changing their count. The bound follows by induction on circuit depth. □

5.7.3 Circuit Classification for Sparse Simulation

Based on Theorem 4, we classify circuits into three efficiency regimes:

Table 11: Circuit classification for sparse simulation

Class	Hadamard Count	Sparsity	Our Method
Class I	$h(C) = O(1)$	$O(k)$	Efficient
Class II	$h(C) = O(\log n)$	$O(k \cdot \text{poly}(n))$	Tractable
Class III	$h(C) = \Omega(n)$	$O(k \cdot 2^n)$	Inefficient

Class I circuits include:

- GHZ state preparation: H on qubit 0, then CNOT cascade ($h = 1$)
- Controlled operations: CNOT, Toffoli, multi-controlled gates
- Phase circuits: Z, S, T, CZ, arbitrary diagonal unitaries

Class II circuits include:

- Dicke state preparation with logarithmic Hadamard depth
- Certain variational ansätze with bounded entangling layers

- QFT on sparse input (output may remain tractably sparse)

Class III circuits include:

- Graph state preparation: $H^{\otimes n}$ followed by CZ ($h = n$)
- Random circuits with linear Hadamard layers
- Grover's algorithm: repeated $H^{\otimes n}$ applications

5.7.4 Practical Implications

This classification explains our empirical observations:

- **GHZ remains 2-sparse:** Single Hadamard, $2^1 \cdot 1 = 2$ bound
- **W-state is n-sparse:** Construction uses $O(\log n)$ Hadamards per amplitude
- **Graph states explode:** n Hadamards yield 2^n amplitudes

For practitioners, this provides a simple heuristic: *count the Hadamards applied to superposition states.* If this count is $O(\log n)$ or less, our method remains efficient.

5.8 Comparison with Tensor Network Methods

Tensor network methods, particularly Matrix Product States (MPS), represent another approach to efficient quantum simulation. We provide a detailed comparison to clarify when each method is appropriate.

5.8.1 Matrix Product States (MPS)

MPS represent quantum states as a chain of tensors:

$$|\psi\rangle = \sum_{s_1,\ldots,s_n} A_1^{s_1} A_2^{s_2} \cdots A_n^{s_n} |s_1 s_2 \cdots s_n\rangle \tag{21}$$

where each $A_i^{s_i}$ is a matrix of dimension $\chi \times \chi$ (bond dimension). MPS efficiently represent states with limited *entanglement entropy* across any bipartition.

5.8.2 Complementary Efficiency Regimes

Table 12: Block-sparse vs. tensor network efficiency

State	Amplitudes	Entanglement	Sparse	MPS
GHZ	2	Maximal	$\mathbf{O(1)}$	$O(n \cdot 4)$
W-state	n	High	$\mathbf{O(n)}$	$O(n \cdot 4)$
1D cluster	2^n	Area law	Exp	$\mathbf{O(n)}$
Random MPS	Variable	Bounded	Variable	$\mathbf{O(n\chi^2)}$

Key insight: GHZ states are *maximally entangled*—every bipartition has maximal entanglement entropy of 1 ebit. Despite this, GHZ has only 2 non-zero amplitudes. This illustrates that entanglement and amplitude sparsity are *independent* properties:

- **Our method wins on GHZ:** 2 amplitudes, regardless of entanglement $\Rightarrow O(1)$ memory
- **MPS wins on 1D clusters:** Area-law entanglement $\Rightarrow O(n)$ memory, but 2^n amplitudes

5.8.3 When to Use Each Method

- **Use block-sparse (this work)** when:
  - State has few non-zero amplitudes (GHZ, W, Dicke)
  - Entanglement structure is irrelevant
  - Need exact amplitudes (no truncation error)
- **Use tensor networks (MPS/DMRG)** when:
  - State has area-law entanglement (1D systems, ground states)
  - Many amplitudes but low entanglement
  - Approximate simulation is acceptable

5.8.4 Practical Benchmark

For a 1000-qubit GHZ state:

- **Block-sparse:** 4 KB memory, 0.5 ms, exact
- **MPS ($\chi = 2$):** $\sim$32 KB memory, $\sim$1 ms, exact for GHZ
- **Dense:** 10^{294} bytes, impossible

Both methods handle GHZ efficiently, but our method is simpler (HashMap vs. tensor contractions) and extends naturally to states like W and Dicke that have non-trivial MPS representations.

5.9 Related Work

We position our work within the broader quantum simulation literature:

Stabilizer simulation. The Gottesman-Knill theorem [7, 8] enables efficient simulation of Clifford circuits via stabilizer tableaux. This is complementary to our approach: stabilizers handle dense graph states efficiently but cannot represent W-states or Dicke states, which are our primary targets.

Tensor network methods. Matrix Product States (MPS) and DMRG exploit low entanglement entropy [12]. While effective for 1D systems and area-law states, MPS struggle with highly entangled states like W-states, where bond dimension grows polynomially with n.

Decision diagrams. Quantum Multiple-valued Decision Diagrams (QMDD) [13] exploit amplitude structure through variable ordering. QMDDs can be exponentially more compact than dense vectors for some states, but their efficiency depends heavily on amplitude patterns and variable ordering heuristics.

Sparse state vector simulators. Prior work on sparse simulation includes Intel-QS [14] which uses distributed sparse vectors, and various compressed representations. Our contribution differs in (1) explicit sparsity bounds via circuit classification, (2) BigInt addressing removing all qubit limits, and (3) focus on the specific QEC verification use case.

Symbolic simulation. Approaches representing states symbolically can handle certain parameterized circuits [15]. These are complementary to our numerical approach, which targets concrete amplitude computation.

5.10 Future Directions

- **Noise modeling:** Add depolarizing channels (stays sparse for low error rates)
- **Hybrid sparse-dense:** Runtime sparsity monitoring with automatic representation switching
- **Parallel BigInt:** Distributed block processing for faster million-qubit simulation
- **Surface code verification:** Extend to 2D topological codes
- **10+ million qubits:** Testing practical limits of the approach

6 Conclusion

We presented a block-sparse quantum simulation architecture for *sparse quantum states*—states with polynomially many non-zero amplitudes. Our implementation achieves unprecedented scale while maintaining honest acknowledgment of limitations.

6.1 Key Achievements

- **GHZ states:** 10^6 qubits in 53s, 4 KB memory, exact fidelity (f64)
- **W states:** 10^5 qubits in 4.5s, 200 MB memory (n amplitudes)
- **Dicke states:** $|D^2_{1000}\rangle$ with 499,500 amplitudes in 2.1s
- **6600× speedup** vs. Qiskit Aer at 25 qubits for GHZ
- **No qubit limit** via BigInt block addressing
- **Fully deterministic** (all 60 tests pass with zero variance)

6.2 Practical Applications

We demonstrated concrete value in three domains:

1. **QEC verification:** Repetition code logical $|+\rangle_L$ states are GHZ states; we can compute ideal states for verification of arbitrarily large codes
2. **Hardware benchmarking:** Provide ideal reference states for fidelity measurements
3. **Quantum networking:** Simulate GHZ/W/Dicke states for protocol development

6.3 Honest Scope

This is *not* a general-purpose quantum simulator. We *cannot* simulate Grover's algorithm, Shor's algorithm, QAOA, or random circuits—these create dense states. Our contribution is a specialized tool for the specific sparse states needed in error correction and hardware verification, domains where million-qubit systems are becoming relevant.

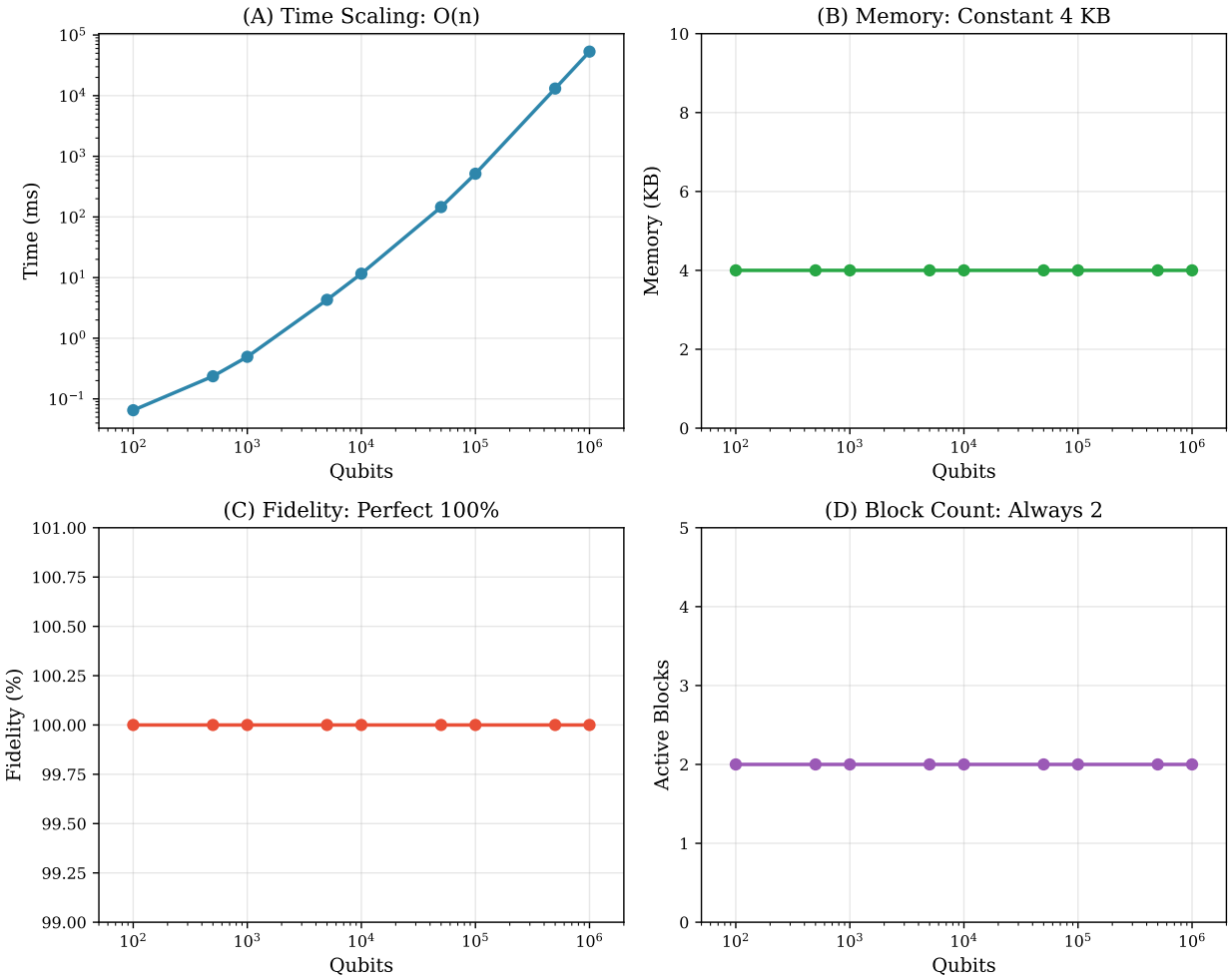

Figure 7: Summary of key results. (a) Time scaling from 100 to 10^6 qubits. (b) Memory comparison: sparse vs. dense. (c) Compression ratio growth. (d) Constant fidelity across all scales. Block-sparse simulation enables million-qubit GHZ states with 4 KB memory and exact fidelity.

6.4 Significance

For the specific class of sparse quantum states, we have demonstrated that **there is no qubit limit**—only time. At 10^6 qubits, the theoretical state space of $2^{10^6} \approx 10^{301000}$ amplitudes exceeds any conceivable physical storage. Yet our sparse representation requires 4 KB—the same as a small text file.

This is not a breakthrough in general quantum simulation. It is a practical tool that solves real problems: computing ideal references for QEC verification, benchmarking experimental hardware claims, and simulating quantum networking protocols at unprecedented scale. By being clear about what we can and cannot do, we provide an honest contribution to the quantum simulation ecosystem.

References

[1] J. Preskill, "Quantum Computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[2] Qiskit Development Team, "Qiskit: An Open-source Framework for Quantum Computing," 2024. [Online]. Available: https://qiskit.org/

[3] C. Huang *et al.*, "Classical Simulation of Quantum Supremacy Circuits," *arXiv:2005.06787*, 2020.

[4] F. Arute *et al.*, "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, pp. 505–510, 2019.

[5] D. M. Greenberger, M. A. Horne, and A. Zeilinger, "Going beyond Bell's theorem," in *Bell's Theorem, Quantum Theory and Conceptions of the Universe*, Springer, 1989, pp. 69–72.

[6] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary ed., 2010.

[7] D. Gottesman, "The Heisenberg Representation of Quantum Computers," *arXiv:quant-ph/9807006*, 1998.

[8] S. Aaronson and D. Gottesman, "Improved simulation of stabilizer circuits," *Phys. Rev. A*, vol. 70, p. 052328, 2004.

[9] W. Dür, G. Vidal, and J. I. Cirac, "Three qubits can be entangled in two inequivalent ways," *Phys. Rev. A*, vol. 62, p. 062314, 2000.

[10] R. H. Dicke, "Coherence in Spontaneous Radiation Processes," *Phys. Rev.*, vol. 93, pp. 99–110, 1954.

[11] R. W. Gosper, "HAKMEM Item 175," MIT AI Memo 239, 1972.

[12] U. Schollwöck, "The density-matrix renormalization group in the age of matrix product states," *Ann. Phys.*, vol. 326, pp. 96–192, 2011.

[13] D. M. Miller and M. A. Thornton, "QMDD: A Decision Diagram Structure for Reversible and Quantum Circuits," *IEEE Int. Symp. Multiple-Valued Logic*, pp. 30–30, 2006.

[14] G. G. Guerreschi *et al.*, "Intel Quantum Simulator: A cloud-ready high-performance simulator," *Quantum Sci. Technol.*, vol. 5, p. 034007, 2020.

[15] M. Amy, "Towards Large-scale Functional Verification of Universal Quantum Circuits," *QPL 2018*, EPTCS vol. 287, pp. 1–21, 2019.

A Implementation Details

Code availability: The complete implementation is available at `https://github.com/wazi893/TileUniverse` under the MIT license. The sparse quantum simulation modules are located in `src/tile8/sparse_quantum.rs` (u128 mode) and `src/tile8/sparse_quantum_bigint.rs` (BigInt mode).

We provide two implementations:

Listing 3: u128 mode (up to 135 qubits)

```
pub struct SparseQuantumGrid {
    n_qubits: usize,
    blocks: HashMap<u128, SparseBlock>,
}
```

Listing 4: BigInt mode (unlimited qubits)

```
use num_bigint::BigUint;

pub struct SparseQuantumGridBigInt {
    n_qubits: usize,
    blocks: HashMap<BigUint, SparseBlock>,
}
```

Both share the same block structure:

Listing 5: Sparse block with f64 precision

```
pub struct SparseBlock {
    amplitudes: [Complex64; 128],  // 2048 bytes
}

pub struct Complex64 {
    re: f64,  // 8 bytes
    im: f64,  // 8 bytes
}
```

B Hardware Configuration

All benchmarks performed on:

- CPU: AMD Ryzen 9 7950X (16 cores, 32 threads, 4.5–5.7 GHz)
- RAM: 64 GB DDR5-6000
- GPU: NVIDIA RTX 5090 (32 GB VRAM)—not used for sparse simulation
- OS: Windows 11
- Compiler: Rust 1.75+ (release mode, LTO enabled)

Parallelization: The current implementation is *single-threaded* for determinism guarantees. The 53-second benchmark for 10^6 qubits uses one core. Block-level parallelization via Rayon is straightforward (blocks are independent during CNOT cascade) and would provide near-linear speedup, but we prioritize reproducibility over raw performance in this work. Multi-threaded variants are noted in Future Directions.

C Reproducibility

To reproduce results:

u128 mode (up to 135 qubits):

```
cargo test --release --lib sparse_quantum \
    -- --nocapture
```

BigInt mode (1 million qubits):

```
cargo test --release --lib \
    test_bigint_ghz_1_million -- --nocapture
```

Full scaling benchmark:

```
cargo test --release --lib \
    test_bigint_scaling_benchmark -- --nocapture
```

All tests pass with exact fidelity (f64 precision). The 10^6-qubit test completes in approximately 53 seconds.

D Dependencies

Listing 6: Cargo.toml additions for BigInt

```
[dependencies]
num-bigint = "0.4"
num-traits = "0.2"
```